PacMan & AI Reinforcement Learning

Innovative AI Applications

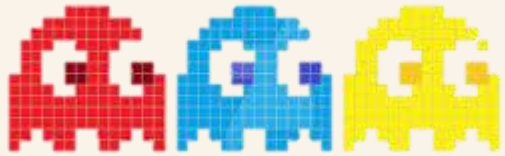
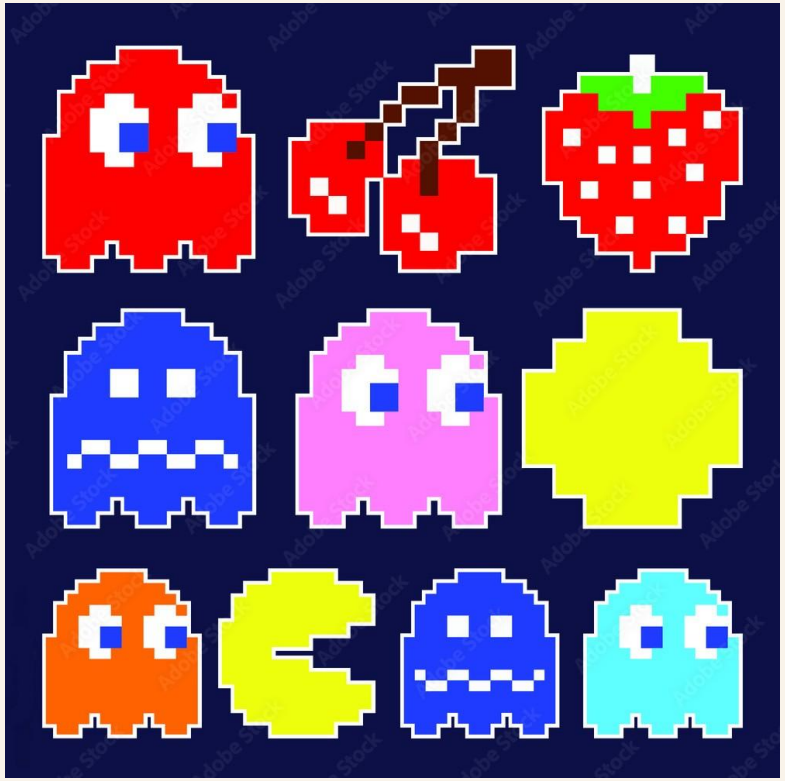




Table of Contents

PacMan And Reinforcement Learning	
01 Problem Background & Description	-How did we come up with this PacMan and Machine Learning idea -Setting up the objective of this project
02 Current Solution and Limitations (how AI help)	-Current innovation on the subject of Pacman -Why our project differs from current innovations
03 Proposed Solution	-Machine Learning Method chose(Q-learning) -Improvement methods in performance
04 Conclusion	-Current checklist of our objectives
05 Future Work	-Future work required to fulfill all objectives

01 Problem Background and Description



01 Problem Background

Pac-Man

→One of the most iconic video games in history

→Developed in the late 1980s

→classical PvE (Player versus Environment) game

→one of the highest-grossing arcade games of all time





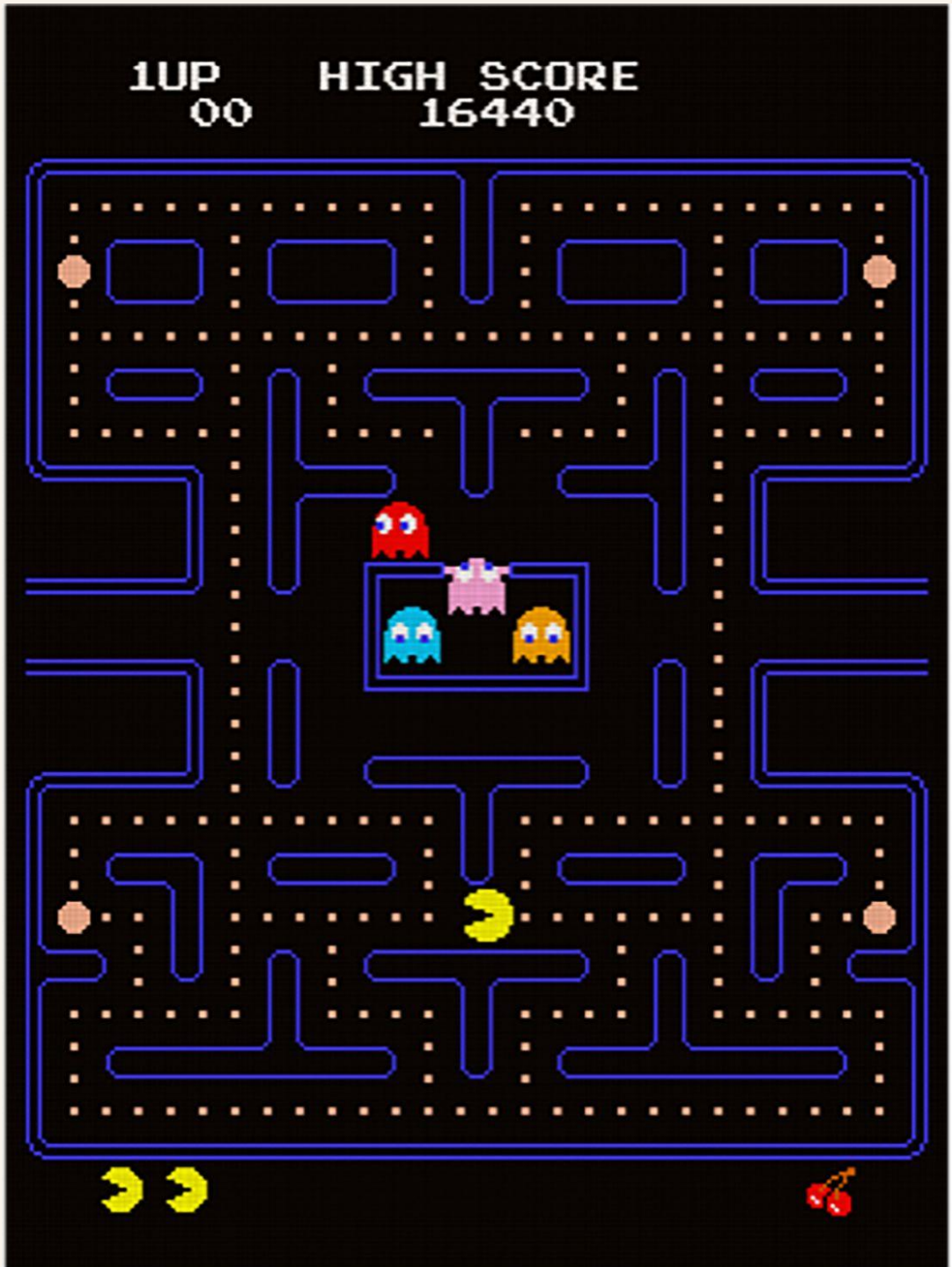
01 Problem Background

Pac-Man

→Controlled by players

→Objectives:

- Eat all the pellets in the maze level in order to level up
- Avoid getting eaten by ghosts 
- Use ‘Cherries’  or ‘Power Pellets’ to defeat the ghosts



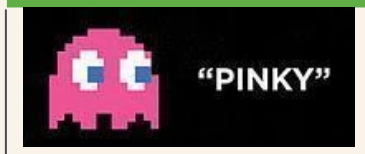
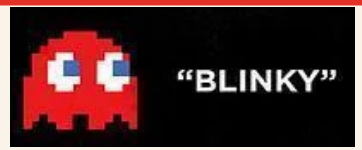
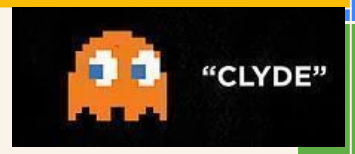
01 Problem Background

Ghosts

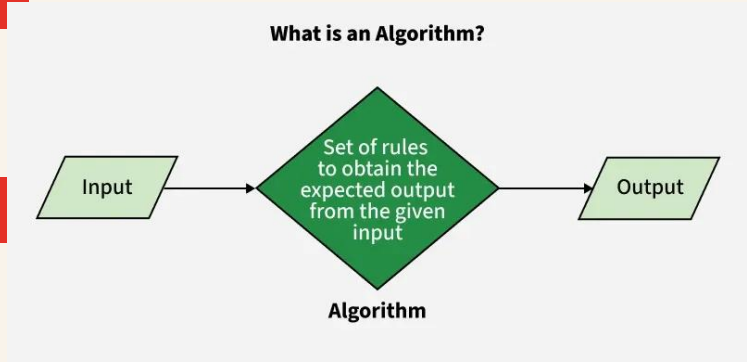
→Defined by fixed algorithms (Bots)

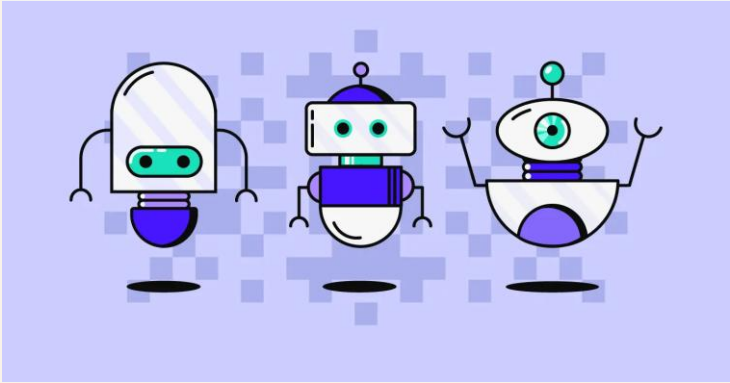
→Plain movements

→Players can easily learn from experience and counter it (knowing ghost characteristics)

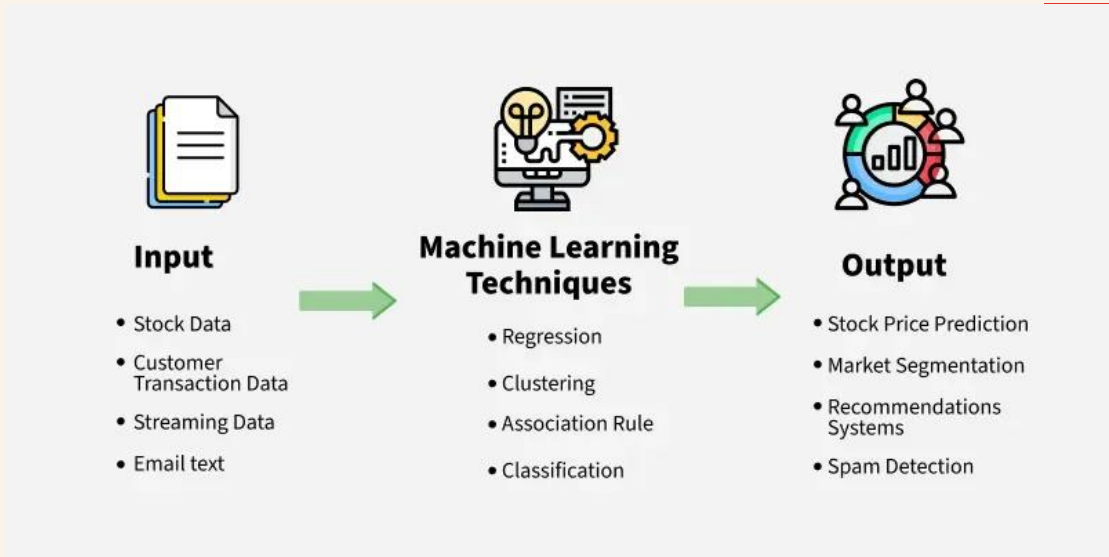
	Pinky	Blinky	Inky	Clyde
				
Chase Mode	Chase after pacman if see pacman within 2 pellets range	Chase directly after Pacman	Chase after pacman under rule	Chase directly after Pacman
Bug/ specialty	Bug: when Pacman faces upward, always position at the left right region of Pacman	Angry mode activate if number of pellets remain are low	Rule: distance with Pacman must be relative to Pinky and Blinky	Timid, when within 8 dot range of Pacman, goes into scatter mode
Scatter spot	Upper-Left	Upper-Right	Lower-Right	Lower-Left

01 Problem Background





VS



Bots

- Algorithm-based programs
- Defined rules by human, inflexible change

AI
(Bots in latest Generation)

- Machine learning models
- Rules/ policy defined by the machine through learning

01 Problem Background

01

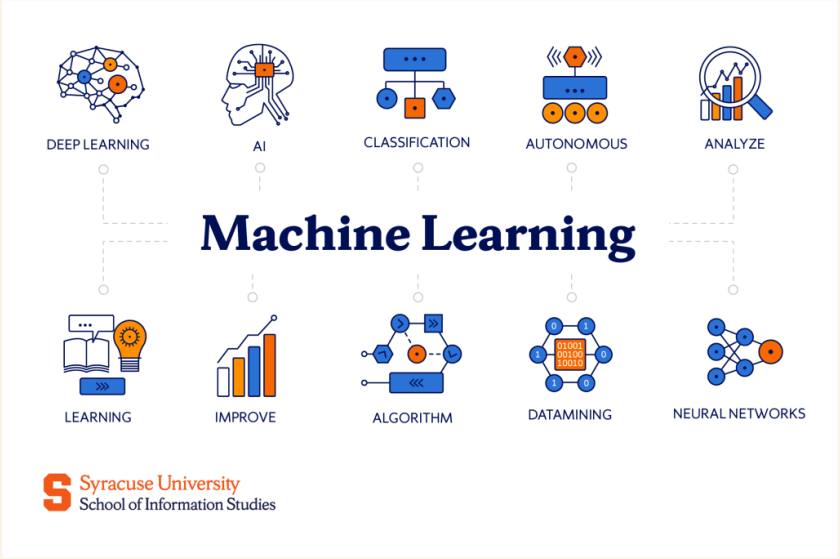


Pac-Man +Ghosts

Arcade game characters, moving in fixed algorithm

Latest Machine Learning Models

Enabling machine to develop their own ‘algorithm’



02

03

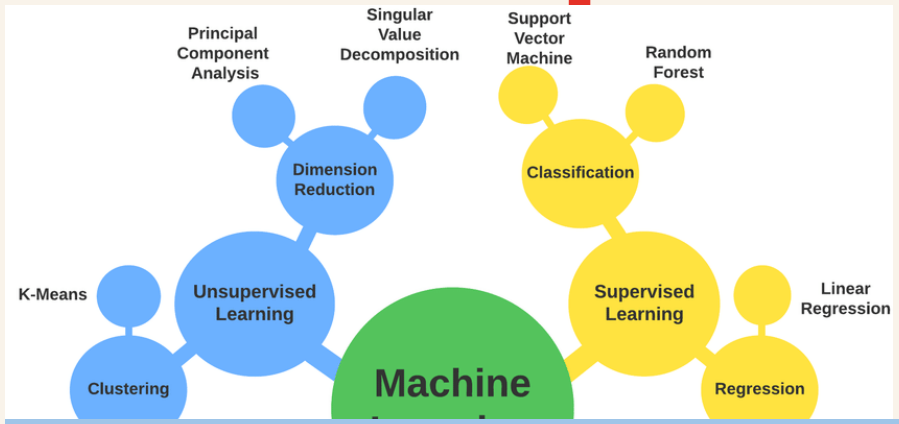
What if we combine both?

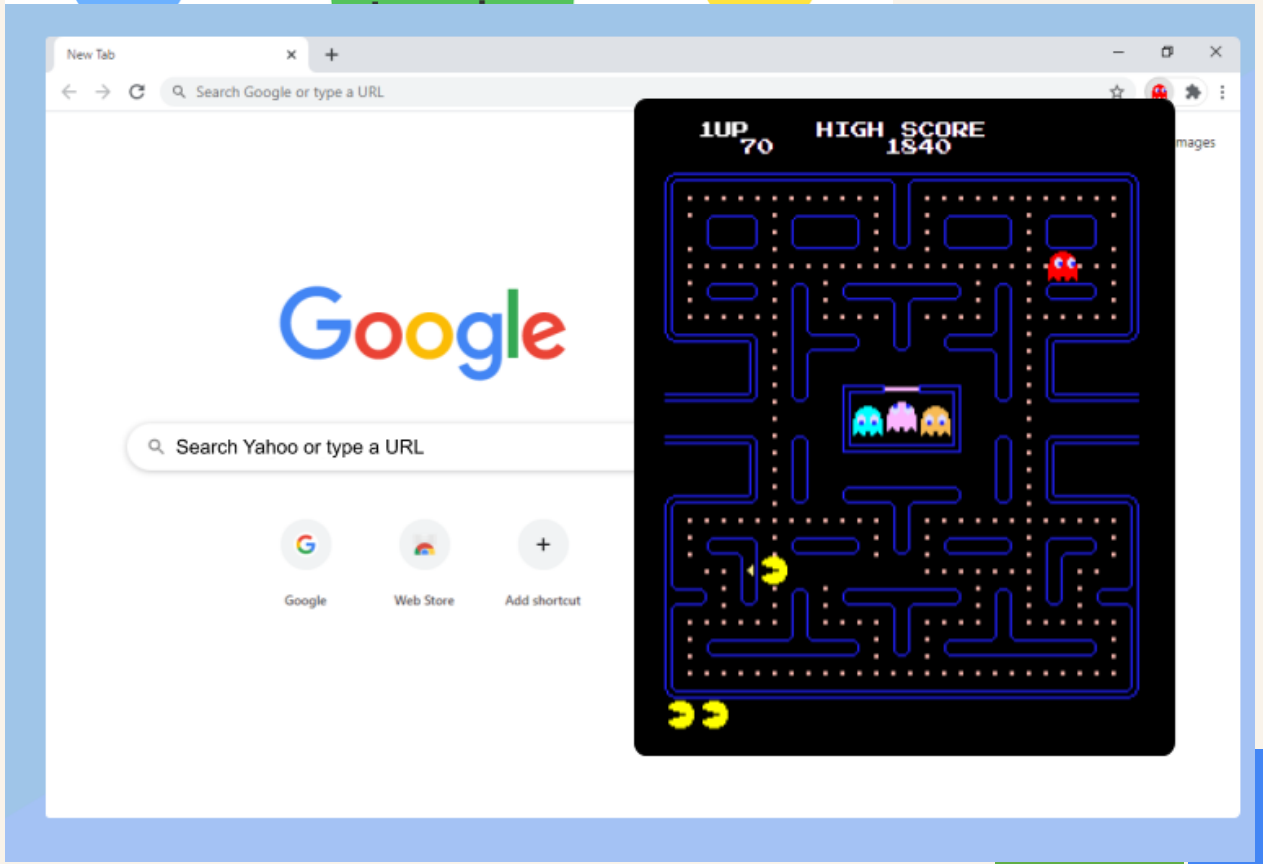
A Pac-Man game with trained AI model ghosts

01 Problem Description

Method:
Find appropriate ML training methods for Pac-Man Characters

-applicable to ghosts and Pacman





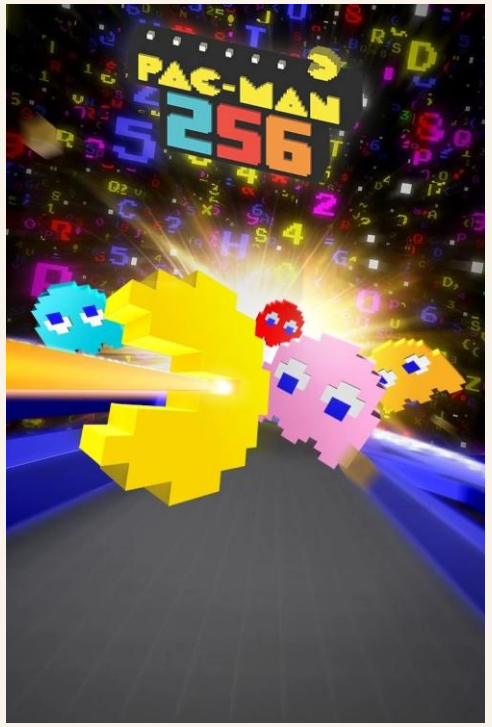
Objective:
-Perform better than an ordinary algorithm-based ghost
- If possible, expand the renewed Pac-Man game to the Internet

02 Current Solution and Limitation

+

How AI help

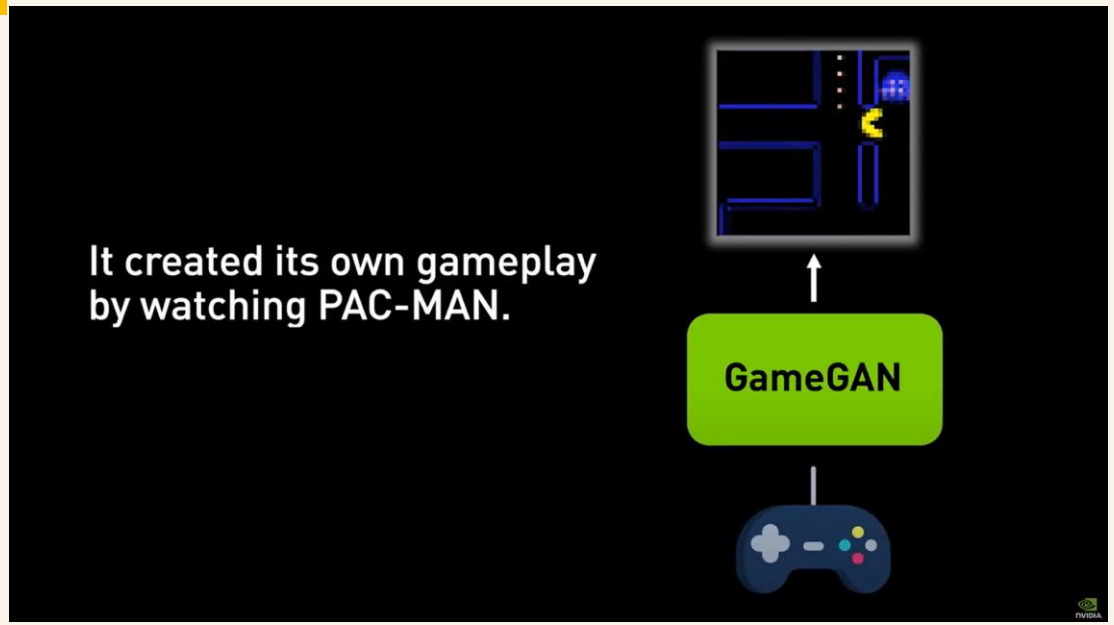




02 Current innovations related to Pacman

Nvidia AI and Pacman

Nvidia generates a complete and fully-functioning Pac-Man game using their new AI model NVIDIA GameGAN [1]



Newer game modes of Pac-Man

New mechanics or game modes implemented in Pac-Man



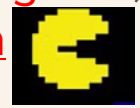
→ Pac-Man 256[2]: endless maze

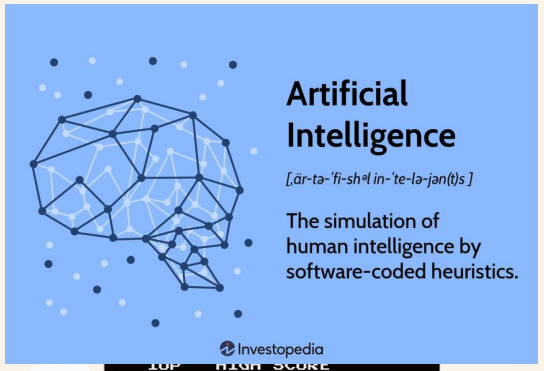


Pac-Man Mega← Tunnel Battle [3]:PvP with online players

02 Limitations of current innovations

Nvidia AI and Pacman

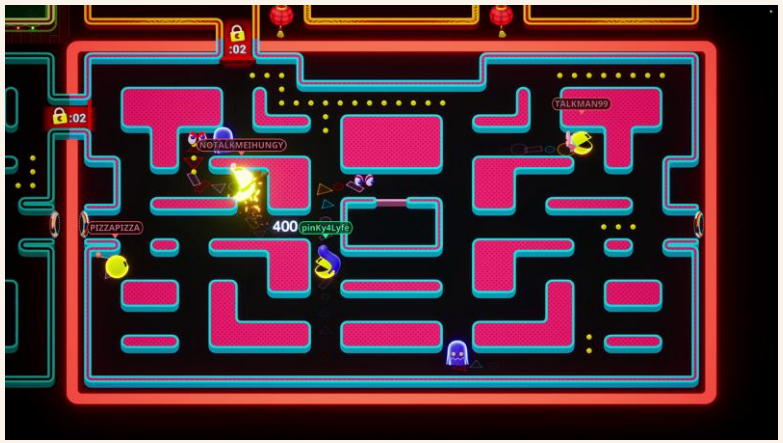
AI is used for generating Pac-Man the game, but not the one playing Pacman 





Newer game modes of Pac-Man

Not much innovative technology involve in the newer Pac-Man games

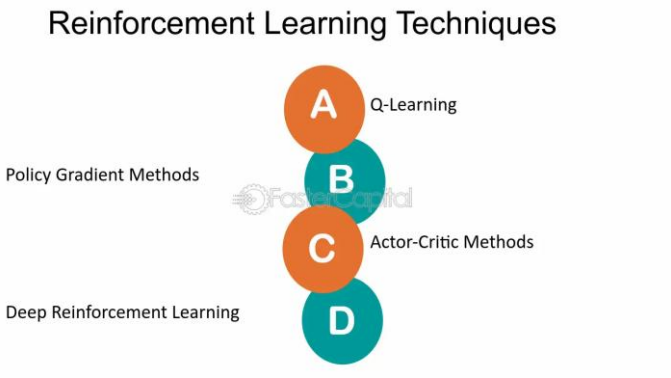


Pac-Man Mega Tunnel Battle
→ enabling online game modes

02 How AI can help/apply to this project

01

Find appropriate machine learning model



→ Reinforcement learning is chosen

02

Train Pac-Man first



→Trained AI Pac-Man wins most of the time
→Means the reinforcement learning method chosen is appropriate for this task

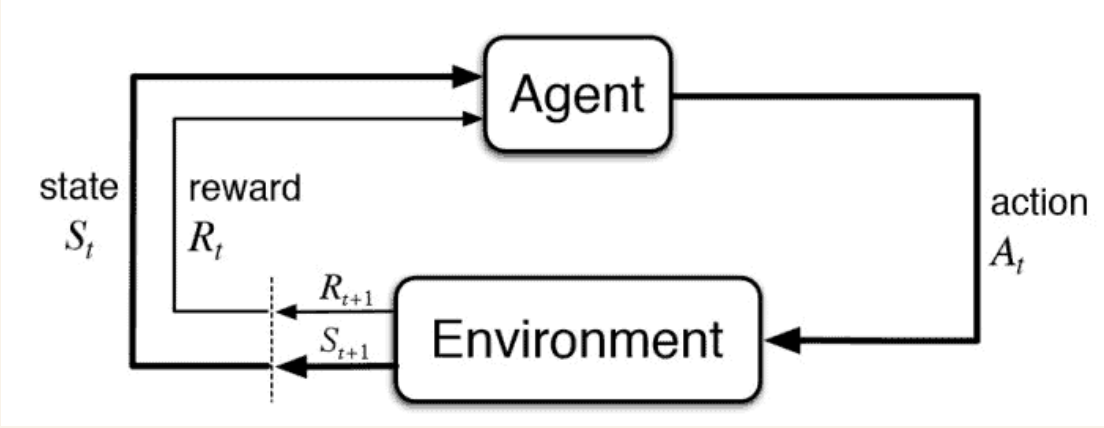
03

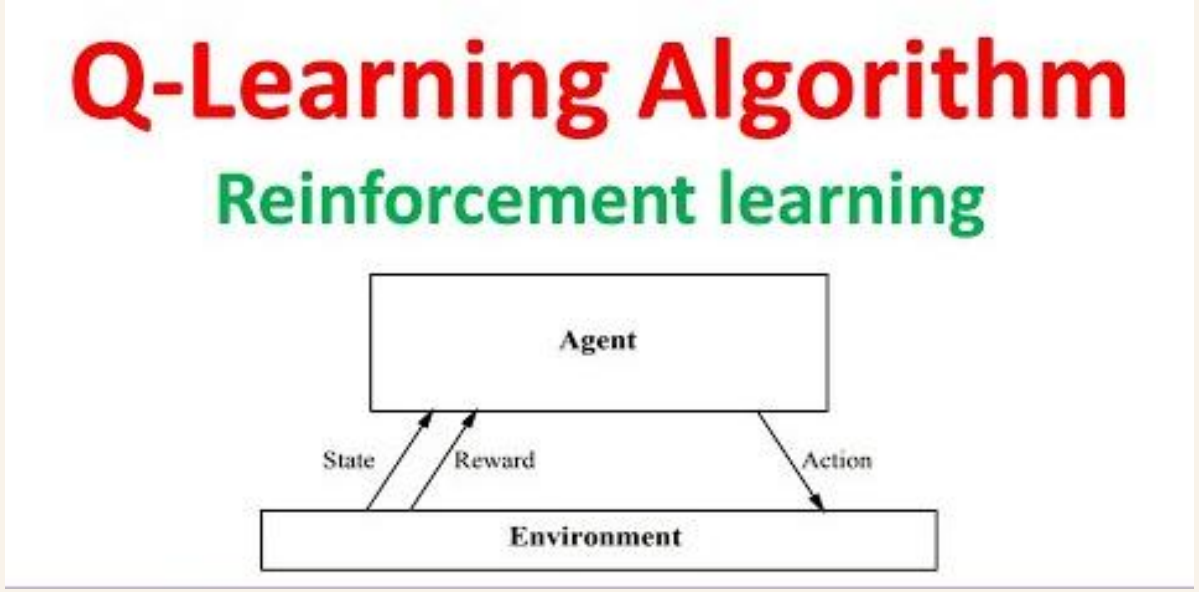
Train 4 ghosts



→Apply the same methods to the 4 ghosts playing against the trained Pacman

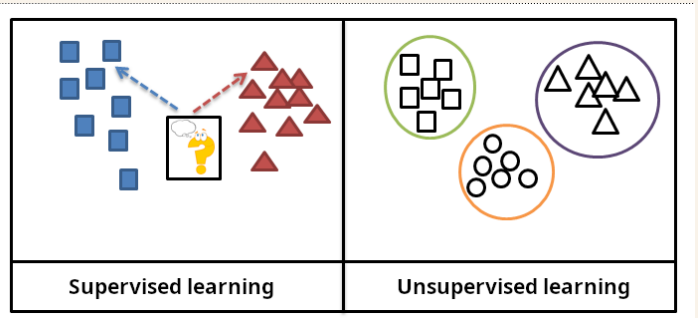
03 Proposed solutions (Q-learning)





03 Proposed solution

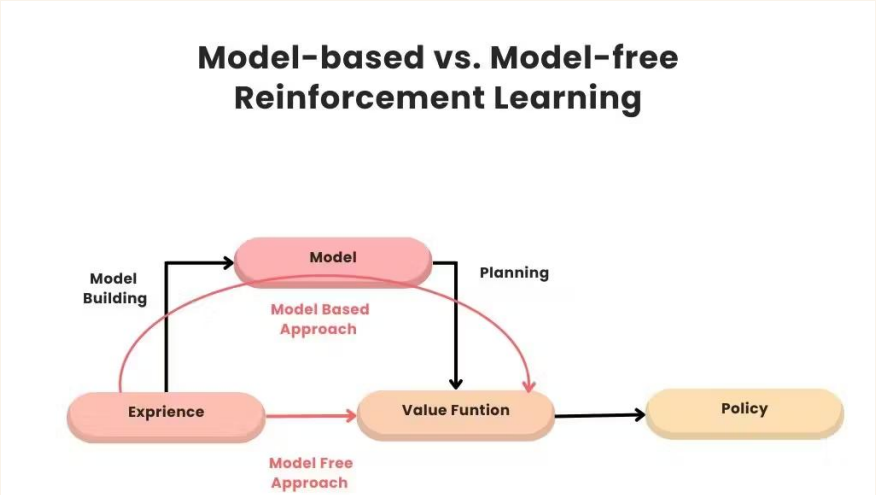
01 Requirements for model



✖ Filtering or categorizing data into labels(X supervised/ unsupervised Learning)

○ Get optimal action/values based on current feedback of results(Reinforcement learning)

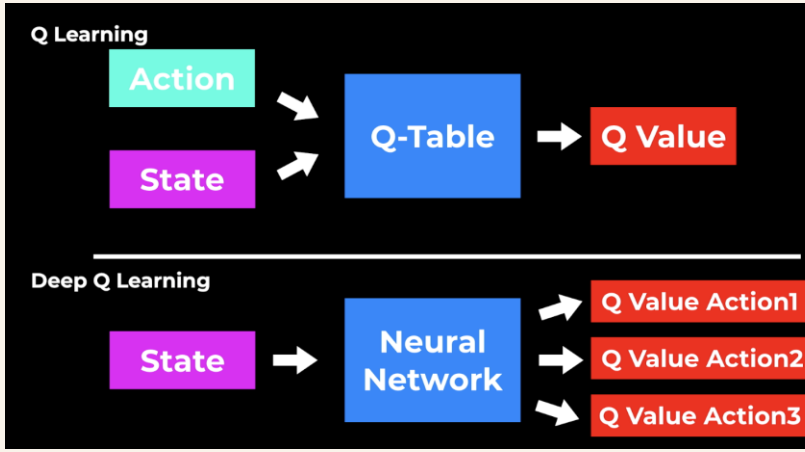
Type of Reinforcement learning



02

→Pac-Man needs to learn through trial and error (doesn't have knowledge about the maze at first)
→Model-free approach

03



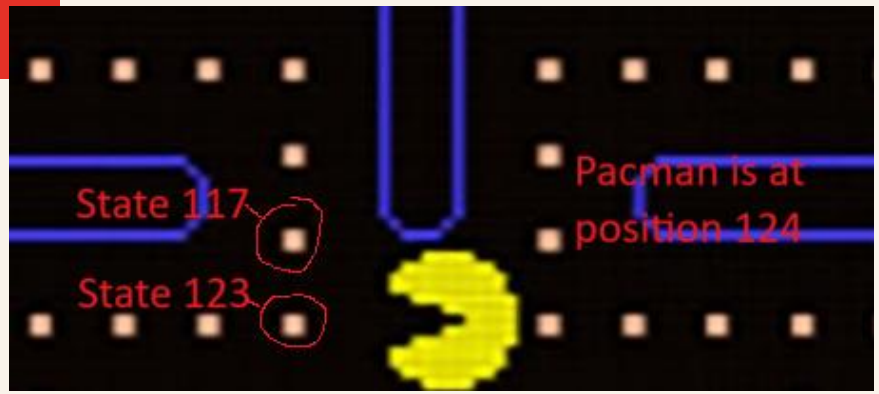
Q-learning[4] selected

03 Proposed solution

Q-table

→ Contain action-state pairs

→ Determine the reward values for taking such action in specific state



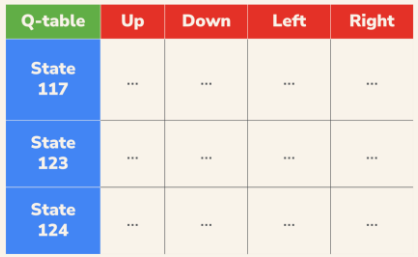
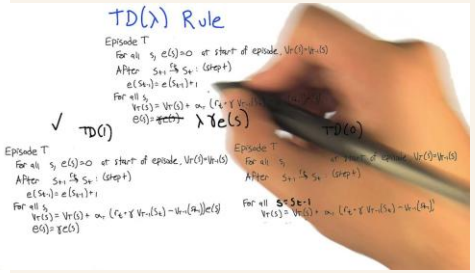
Actions

Q-table	Up	Down	Left	Right
State 117	...	...	...	...
State 123	...	...	...	...
State 124	...	...	...	...

State(Coordinates on the maze)

03 Proposed solution

Q-learning Steps

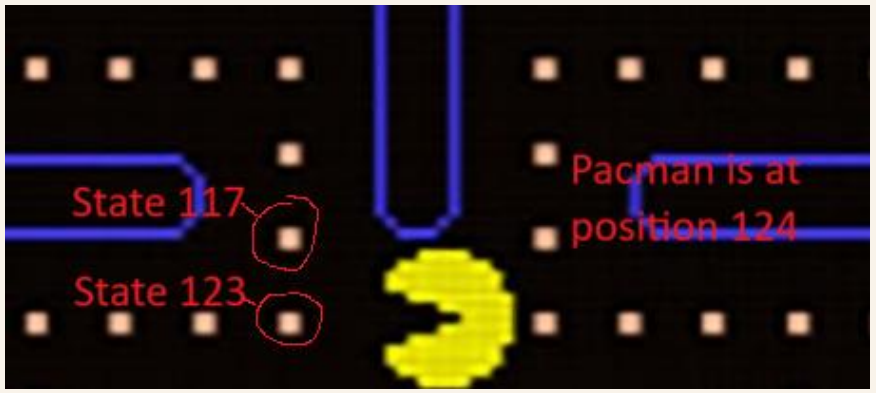
Initialization	Trials	Action and Update	Iteration	Finish episode
				
Start-up	**Trial and error**	**Calculate reward**	**Repeat**	**Complete training**
Setting all Q-table entries to zero	Exploration policy(Epsilon Greedy) applied to agent for selecting actions	After action, observe next state, receive reward, update Q-table for evaluation	Repeat the exploration and update Q-table until the run finish	Repeat the runs by an iterative process (1 episode = 1 run)

03 Proposed solution

Step 1: Initialization

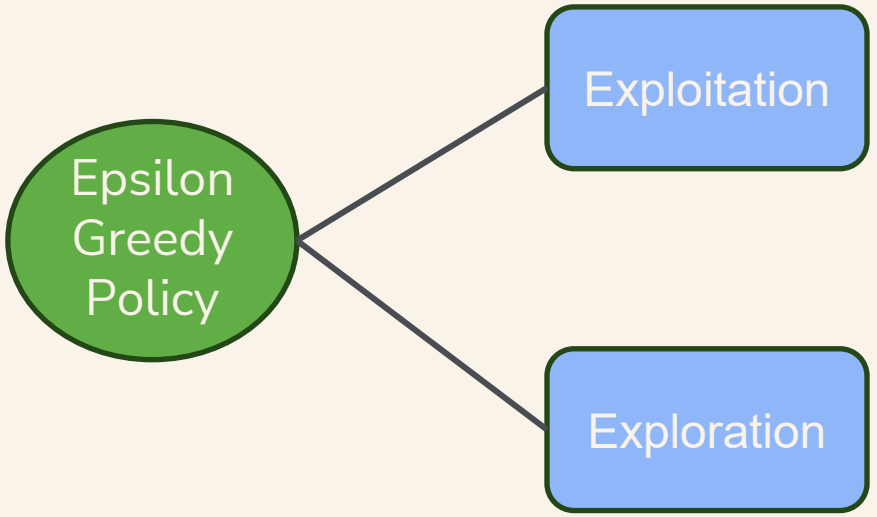
→ Setting all Q-values as zero

Q-table	Up	Down	Left	Right
State 117	0	0	0	0
State 123	0	0	0	0
State 124	0	0	0	0



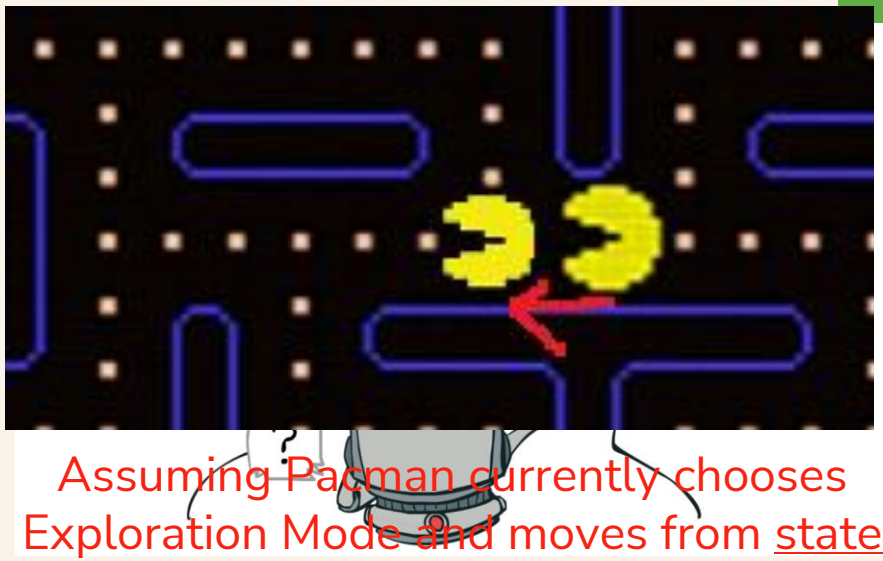
03 Proposed solution

Step 2: Exploration Policy



→Pacman chooses action with the highest Q-value with probability 1−ϵ

→Pacman chooses random action with probability ϵ



Assuming Pacman currently chooses Exploration Mode and moves from state 124 to state123

Advantage:
-Prevent the agent over over-relying on current knowledge
-allow agent to discover new strategies + improve decision making in the long run

03 Proposed solution

Step 3: Action and Update

→ Bellman equation used:

$$Q(s,a)_{observed} = R(s,a) + \gamma \max_a Q(s',a)$$

- $Q(s,a)_{observed}$: Q-value for such state-action pair
- $R(s,a)$: Immediate reward for current state-action
- γ: Discount Factor
- $\max_a Q(s',a)$: Max Q-value for next state

Q-table	Up	Down	Left	Right
State 117	-0.5	1	1.3	2.4
State 123	0.3	0.5	1.2	0.75
State 124	-1.4	1.2	0.6	1.5



Assuming the Pacman already run a couple actions (some Q-values set up)

Step 3: Action and Update

Reward table→

Pacman (state 117) : -1	…	…
Pacman (state 123) : -1	Pacman(state 124): -1	…
…	…	…

→ Bellman equation used:

→ $Q(\text{state124, Left})_{\text{observed}} = R(\text{state } 123) + \gamma \max_a Q(\text{state } 123, a)$

→ $Q(\text{state124, Left})_{\text{observed}} = -1 + 0.1 * 1.2 = -0.88$

Action-value calculation complete for taking such action at this state



Q-table	Up	Down	Left	Right
State 117	-0.5	1	1.3	2.4
State 123	0.3	0.5	1.2	0.75
State 124	-1.4	1.2	0.6	1.5

Pacman moving from state 124 to 123 by doing action left (given discount factor = 0.1)

03 Proposed solution

Step 3: Action and Update

Current expected Q-value

→*Temporal Difference(TD) error* = $Q(s,a)_{observed} - Q(s,a)_{expected}$

= $R(s,a) + \gamma \max_a Q(s',a) - Q(s,a)_{expected}$

Learning rate

→*TD update equation*: $Q(s,a) \leftarrow Q(s,a) + \alpha TD_error$

$= Q(s,a) \leftarrow Q(s,a) + \alpha(R(s,a) + \gamma \max_a Q(s',a) - Q(s,a)_{expected})$

New Q-value(update)



Q-table	Up	Down	Left	Right
State 117	-0.5	1	1.3	2.4
State 123	0.3	0.5	1.2	0.75
State 124	-1.4	1.2	0.6	1.5

Pacman moving from state 124 to 123 by doing action left (given learning rate = 0.1)

03 Proposed solution

Step 3: Action and Update

→ *TD error* = $Q(\text{state 124, Left})_{\text{observed}} - Q(\text{state 124, Left})_{\text{expected}}$

= -0.88 – 0.6 = -1.48

→*TD update:*

Q(state 124, Left) = Q(state 124, Left)+αTD_error

Q(s,a)= 0.6 + 0.1*(-1.48) = **0.452** (**New Q-value**)

Q-table	Up	Down	Left	Right
State 117	-0.5	1	1.3	2.4
State 123	0.3	0.5	1.2	0.75
State 124	-1.4	1.2	0.6	1.5



Pacman moving from state 124 to 123 by doing action left (given learning rate = 0.1)

03 Proposed solution

Step 4: Repeat iteration

Explore

03 Proposed solution

Step 2: Exploration Policy

Epsilon Greedy Policy

- Exploitation → Pacman chooses action with the highest Q-value with probability 1−ϵ
- Exploration → Pacman chooses random action with probability ϵ



Assuming Pacman currently chooses Exploration Mode and moves from state 124 to state123

Advantage:
-Prevent the agent over over-relying on current knowledge
-allow agent to discover new strategies + improve decision making in the long run

Action

Step 03: Action and Update

Reward table→

Pacman (state 117) : -1	...	...
Pacman (state 123) : -1	Pacman(state 124): -1	...
...	...	...

→ Bellman equation used:

→ $Q(state124, Left)_{observed} = R(state\ 123) + \gamma \max_a Q(state\ 123, a)$

→ $Q(state124, Left)_{observed} = -1 + 0.1 * 1.2 = -0.88$

Action-value calculation complete for taking such action at this state



Q-table	Up	Down	Left	Right
State 117	-0.5	1	1.3	2.4
State 123	0.3	0.5	1.2	0.75
State 124	-1.4	1.2	0.6	1.5

Pacman moving from state 124 to 123 by doing action left (given discount factor = 0.1)

Update

Step 03: Action and Update

→ *TD error* = $Q(state\ 124, Left)_{observed} - Q(state\ 124, Left)_{expected}$
= -0.88 – 0.6 = -1.48

→*TD update:*

Q(state 124, Left) = Q(state 124, Left)+αTD_error

Q(s,a)= 0.6 + 0.1*(-1.48) = **0.452 (New Q-value)**

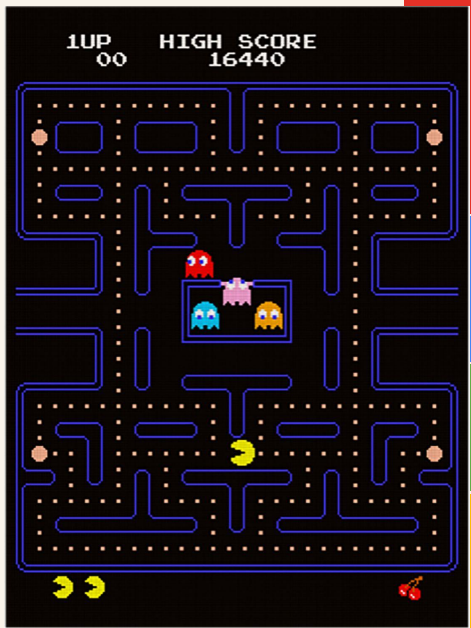


03 Proposed solution

Q-table	Up	Down	Left	Right
State 117	-0.5	1	1.3	2.4
State 123	0.3	0.5	1.2	0.75
State 124	-1.4	1.2	0.6	1.5

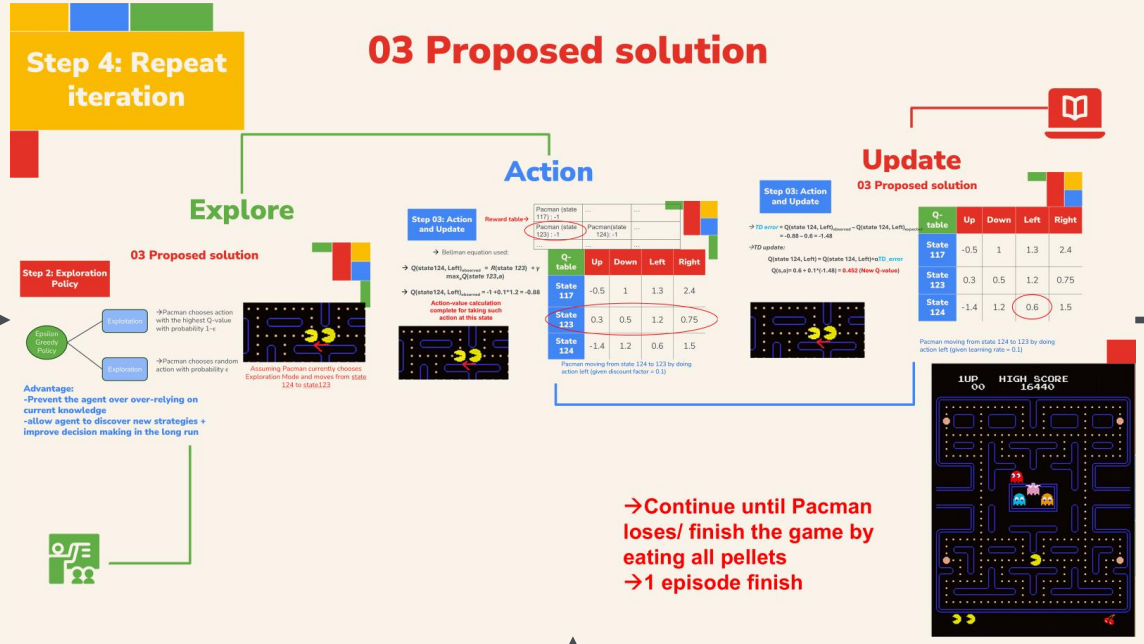
Pacman moving from state 124 to 123 by doing action left (given learning rate = 0.1)

→Continue until Pacman loses/ finish the game by eating all pellets
→1 episode finish



03 Proposed solution

Step 5: Repeat episode



If current episode< episode set

Yes

No

Finish training, ready for test

Start training

03 Proposed solution

Improvement Required

✖ After training, if the maze magnitude increases, the performance of Pacman decreases significantly



○ Scalable performance required

Probable faults with Q-learning in our situation:
→Too many states if the maze magnitude increases (compute 1000 x 1000 pixel Q-entries)

Generalize the Q-entries within a region
→ Approximate Q-learning[5]

03 Proposed solution

Q-table	Up	Down	Left	Right
Region (0,0) to (64,64)	...	...	...	...
Region (65,0) to (128, 128)	...	...	...	...
---	...	...	...	...

Generalizing a region of Q-entires with a feature factor (Weight)

Approximate Q-value function:

$$Q(s,a)=\sum_{i=1}^{n} f_i(s,a)*w_i$$

Weight mapping:

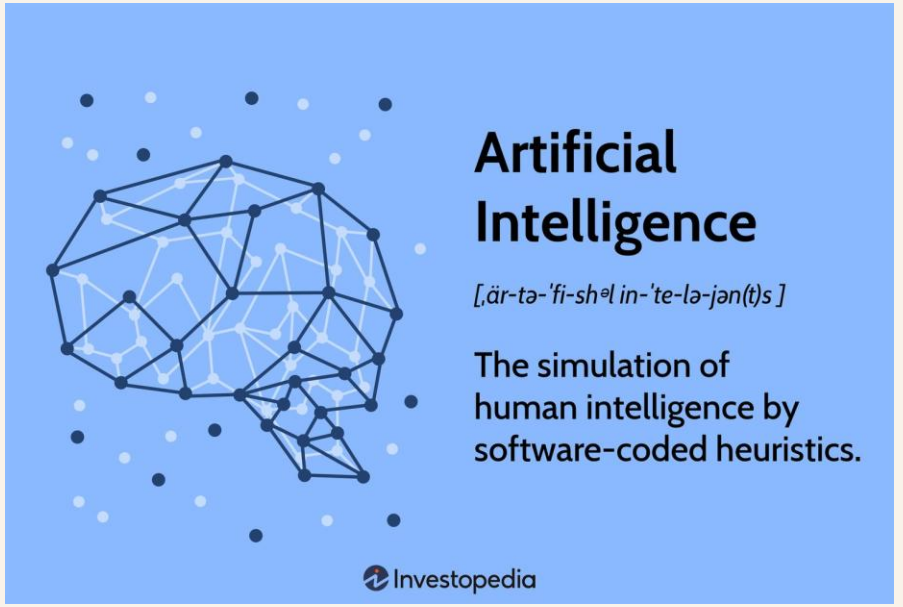
$$w_i \leftarrow w_i+\alpha\cdot difference\cdot f_i(s,a)$$

$$difference= (r + \gamma \max_{a'} Q(s',a'))-Q(s,a)$$

Approximate Q-learning →performance doesn't decay even the maze complexity and magnitude increases

04 Conclusion





04 Conclusion

Find appropriate ML method



Pacman AI agent run smoothly



Improvement to the AI agent



Apply to the 4 ghosts



Trained AI Ghost plays well with AI Pacman



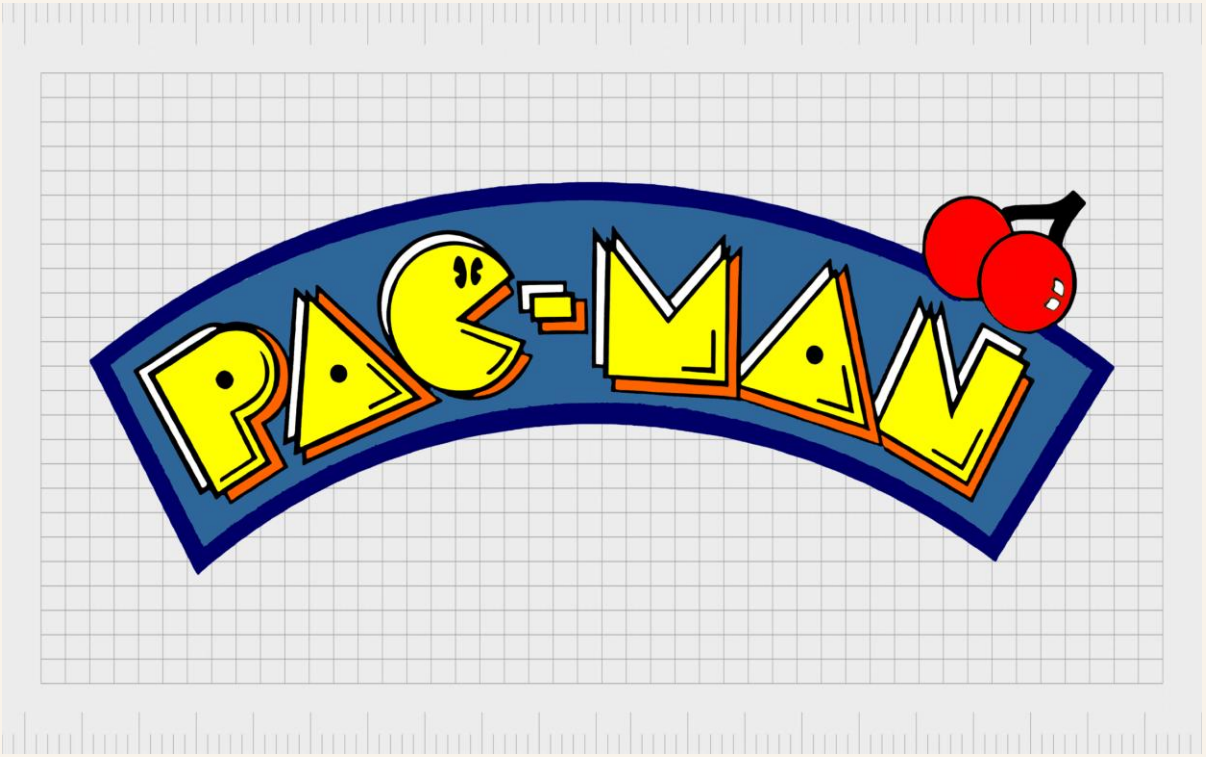
Develop new meachanism for Pacman game (commercialization)



Deploy the game as a website



05 Future Work





05 Future Work

Apply ML algorithm to the 4 ghosts

→Idea in Mind/ Expectation:
Training 4 ghost under the same Q-table, developing same policy

→Allow the ghosts to develop strategies/ work in coordination



05 Future Work

Trained AI Ghost plays well with AI Pacman

→Clearing bugs if any occur

Why is he trying to eat us? We aren't in frightened mode.





Probable situation when Pacman switches through different modes

05 Future Work

Develop new meachanism for Pacman game (commercialization)

→Devloping some new mechanics to commercialize the game
→attract players attention to this new game





→ Mechanic 1: AI mode pellets, allowing the AI to control Pacman for a while

05 Future Work

Develop new meachanism for Pacman game (commercialization)

→Mechanic 2:

Level 1-5: Ghost agents undergo 100 training episodes (less clever state)

Level 6-10: AI analyzes players' skill level based on level 1-5, trains ghost agents with respective training episodes

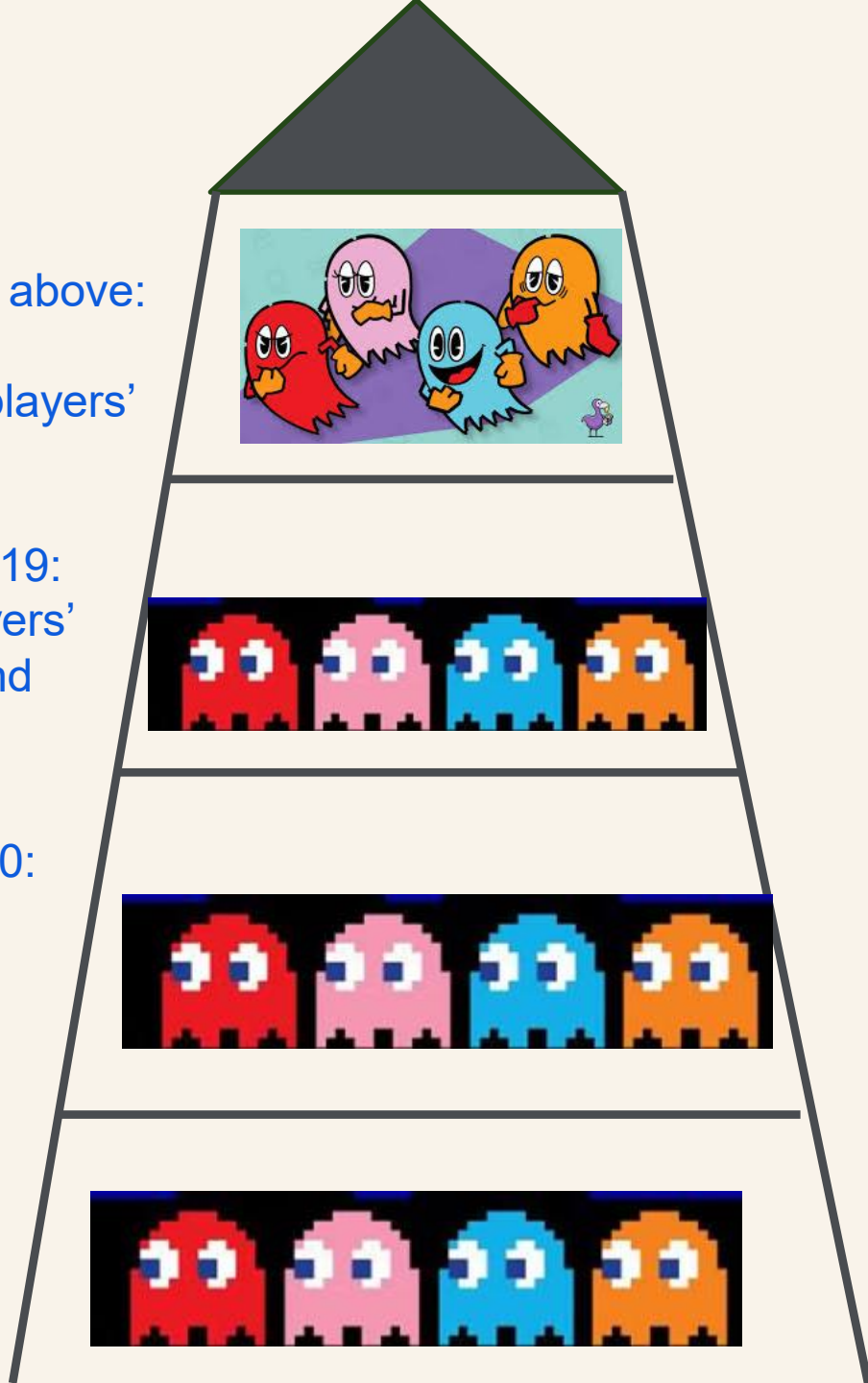
Level 11: Apply the trained ghost agents to the game, also undergo Q-learning training for ghosts using players' gameplay(analyze their behaviours)

→ Level 20/ above: Eventually countering players' strategies

→ Level 11 -19: Analyze players' behaviour and evolve

→ Level 6-10: Undergoing training

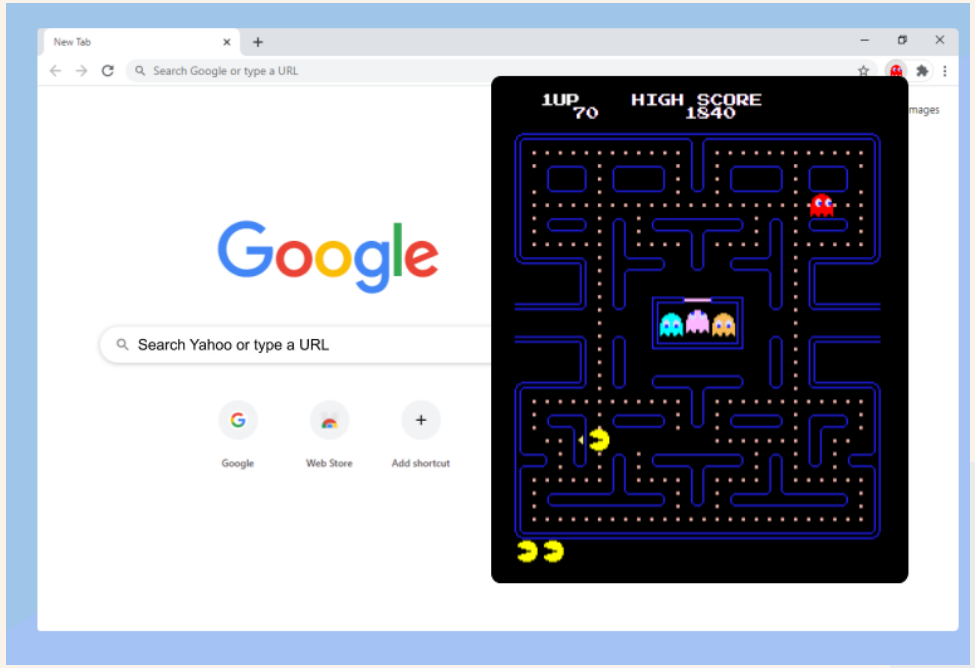
→ Level 1-5: Less clever ghosts



05 Future Work

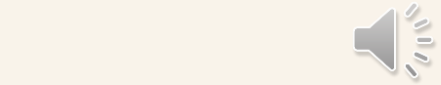
Deploy the game as a website

→Deploying the game for people to play





Thank you for listening.







PowerPoint template credits to SlidesGo.

Reference and Acknowledgement:

[1]: 40 Years on, PAC-MAN Recreated with AI by NVIDIA Researchers, https://blogs.nvidia.com/blog/gamegan-research-pacman-anniversary/
[2]: Pacman 256 fandom, https://pacman.fandom.com/wiki/Pac-Man_256
[3]: Pacman tunnel Mega Tunnel Battle fandom, https://pacman.fandom.com/wiki/Pac-Man_Mega_Tunnel_Battle
[4]: Q-learning in Reinforcement Learning, https://www.geeksforgeeks.org/q-learning-in-python/
[5]: Approximate Q-learning concept, https://www.youtube.com/watch?v=Vky0WVh_FSk&t=931s
[6]: Reference for Pacman and reinforcement learning, https://berkeleyai.github.io/cs188-website/project3.html